

NHẬP MÔN KHOA HỌC DỮ LIỆU

#3. Ngôn ngữ lập trình Python (2)

**INTRODUCTION TO
DATA SCIENCE**

2025 - 2026

- 1. Kiểu dữ liệu & phép toán liên quan**
- 2. Cấu trúc rẽ nhánh**
- 3. Vòng lặp**
- 4. Hàm**
- 5. Bài tập**

Phần 1

Kiểu dữ liệu & phép toán liên quan





Kiểu số

• Python viết số nguyên theo nhiều hệ cơ số

- `A = 1234` # hệ cơ số 10
- `B = 0xAF1` # hệ cơ số 16
- `C = 0o772` # hệ cơ số 8
- `D = 0b1001` # hệ cơ số 2

• Chuyển đổi từ số nguyên thành string ở các hệ cơ số khác nhau

- `K = str(1234)` # chuyển thành str ở hệ cơ số 10
- `L = hex(1234)` # chuyển thành str ở hệ cơ số 16
- `M = oct(1234)` # chuyển thành str ở hệ cơ số 8
- `N = bin(1234)` # chuyển thành str ở hệ cơ số 2



Kiểu số

- Từ python 3, số nguyên không có giới hạn số chữ số
- Số thực (**float**) trong python có thể viết kiểu thông thường hoặc dạng khoa học
 - $X = 12.34$
 - $Y = 314.15279e-2$ #dạng số nguyên và phần mũ 10
- Python hỗ trợ kiểu số phức, với chữ j đại diện cho phần ảo
 - $A = 3+4j$
 - $B = 2-2j$
 - `print(A+B)` #sẽ in ra $(5 + 2j)$



Phép toán

• Python hỗ trợ nhiều phép toán số, logic, so sánh và phép toán bit

- Các phép toán thông thường: **+, -, *, %, ****
- Python có 2 phép chia:
 - Chia đúng (/): `10/3` **#3. 33333333333333335**
 - Chia nguyên (//): `10//3` **# 3 (nhanh hơn phép /)**
- Các phép logic: **and, or, not**
 - Python **không có phép xor** logic, trường hợp muốn tính phép xor thì thay bằng phép so sánh khác (`bool(a) != bool(b)`)
- Các phép so sánh: **<, <=, >, >=, !=, ==**
- Các phép toán bit: **&, |, ^, ~, <<, >>**
- Phép kiểm tra tập **(in, not in): 1 in [1, 2, 3]**

Phần 2

Cấu trúc rẽ nhánh

Cấu trúc if-else

SƠ ĐỒ Rẽ NHÁNH 'elif' TRONG PYTHON: MINH HỌA CƠ CHẾ ĐIỀU KHIỂN ĐA TẦNG

1. CẤU TRÚC 'if' ĐƠN GIẢN

```
if expression:
    # If-block
```

3. CẤU TRÚC 'elif' ĐA TẦNG (VỚI else)

```
if expression:
    # If-block
elif 2-expression:
    # 2-if-block
else:
    # else-block
```

4. CẤU TRÚC 'if-else'

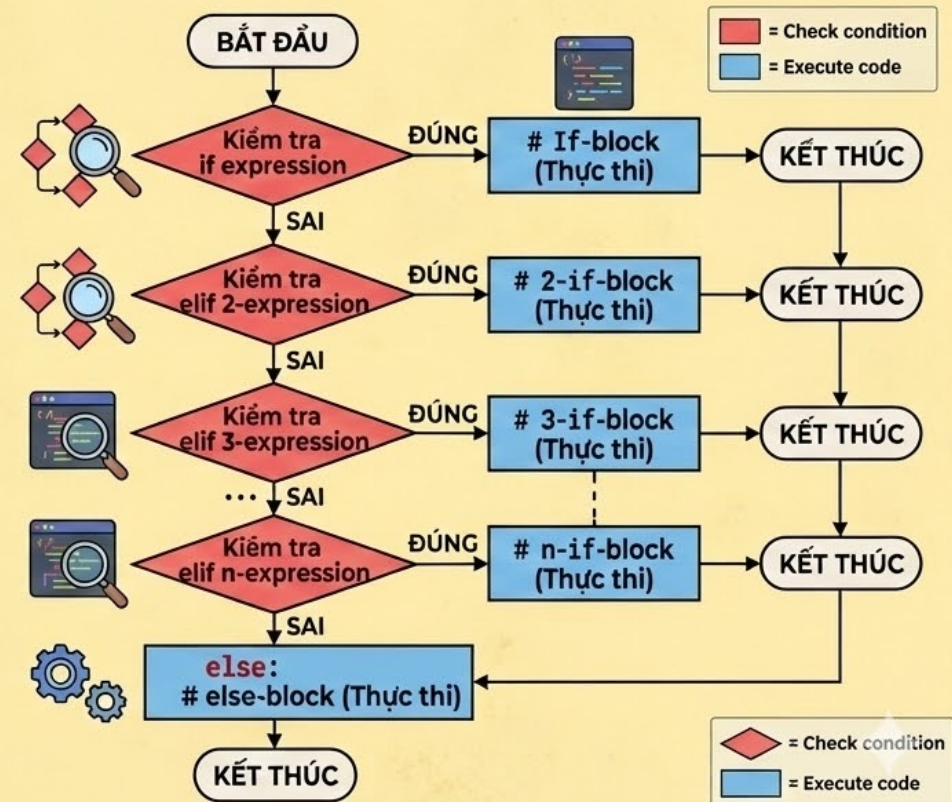
```
if expression:
    # If-block
else:
    # else-block
```

2. CẤU TRÚC 'elif' ĐA TẦNG

```
if expression:
    # If-block
elif 2-expression:
    # 2-if-block
elif 3-expression:
    # 3-if-block
...
elif n-expression:
    # n-if-block
```

4. CẤU TRÚC HOÀN CHỈNH (if-elif-else)

```
if expression:
    # If-block
elif 2-expression: # 2-if-block
...
elif n-expression: # n-if-block
else:
    # else-block
```





Chú ý khối mã trong if-else

- **Chú ý: python nhạy cảm với việc viết khối mã**

vd02.py > ...

```
1  name = input("What's your name?")
2  print("Nice to meet you " + name + "!!!")
3  age = int(input("Your age? "))
4  print("You are already", age, "years old, ", name, "!")
5  if age >= 18:
6      print("Đủ tuổi đi bầu cử")
7      if age > 100:
8          print("Huốt tuổi...")
9  else:
10     print("Chưa đủ tuổi")
```



Phép toán 'if'

- Python có cách sử dụng if khá kì cục (theo cách nhìn của những người đã biết lệnh if trong một ngôn ngữ khác)
 - Nhưng cách viết này rất hợp lý xét về mặt ngôn ngữ và cách đọc điều kiện logic
- **Cú pháp: A if <điều-kiện> else B**
 - Giải thích: phép toán trả về A nếu điều-kiện là đúng, ngược lại trả về B
 - Ví dụ: **X = A if A > B else B** #X m=la max của A và B

Phần 3

Vòng lặp



Vòng lặp while

- **Chú ý**

- Lặp while trong python tương đối giống trong các ngôn ngữ khác
- Trong khối lệnh while (lệnh lặp nói chung) có thể dung **continue** hoặc **break** để về đầu hoặc cuối khối lệnh
- Khối “**else**” sẽ được thực hiện sau khi toàn bộ vòng lặp đã chạy xong
 - Khối này sẽ không chạy nếu vòng lặp bị “**break**”

Vòng lặp for



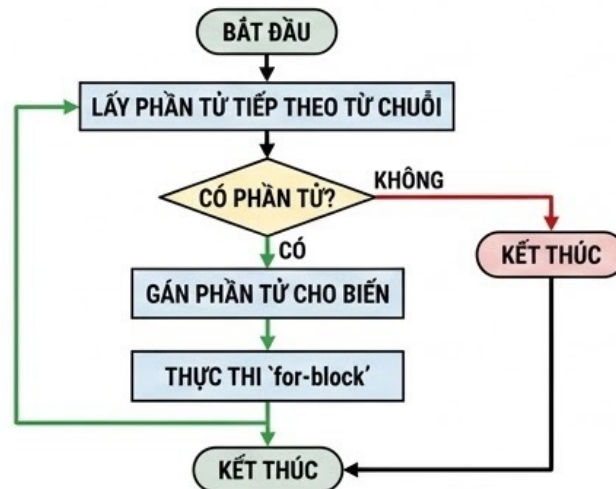
SƠ ĐỒ VÒNG LẶP FOR TRONG PYTHON



1. VÒNG LẶP `for` CƠ BẢN

```
for variable in sequence:
    # for-block
```

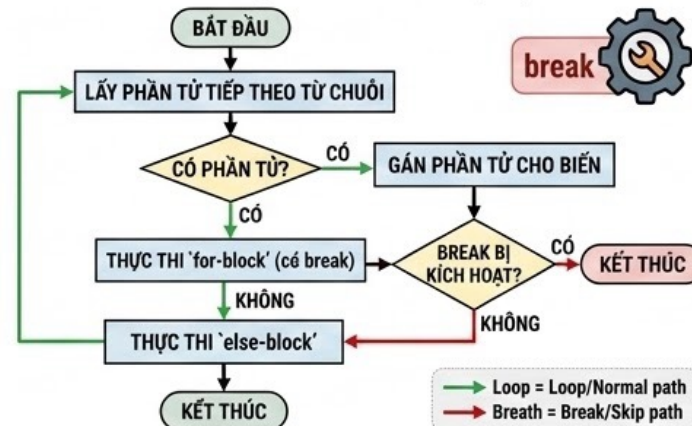
- Vòng lặp `for` được dùng để duyệt các danh sách hoặc chuỗi tuần tự.



2. VÒNG LẶP `for` VỚI `else`

```
for variable in sequence:
    # for-block
else:
    # else-block
```

- Khối `else` sẽ được thực thi khi vòng lặp kết thúc bình thường (không bị ngắt bởi **break**).
- "làm việc tương tự như ở vòng lặp `while`"



Lưu ý: Khối `else` chỉ chạy khi vòng lặp **KHÔNG** bị "break".

3. HÀM `range()` & GIẢI NÉN ĐA BIẾN



Phần range()

Dùng hàm **range(a, b)** để tạo danh sách gồm các số từ a đến b-1, hoặc tổng quát hơn là **range(a, b, c)** trong đó c là bước nhảy.

```
range(1, 5)  →  range(1, 10, 2)
1 → 2 → 3 → 4  →  1 → 3 → 5 → 7 → 9
[1, 2, 3, 4]  →  [1, 3, 5, 7, 9]
```



Giải nén đa biến

```
# Ví dụ: Duyệt danh sách các tuple
pairs = [(1, "A"), (2, "B")]
for num, char in pairs:
    print(f"{num}: {char}")
```

Khi **duyet** một danh sách các tuple, ta có thể giải nén các phần tử trực tiếp vào các biến vòng lặp.

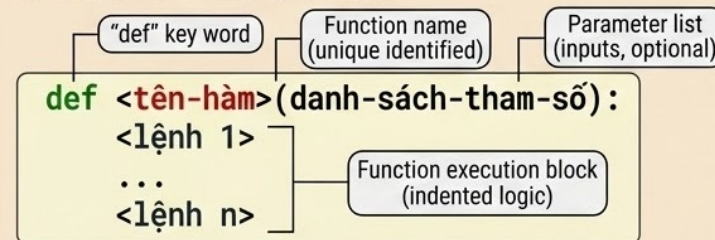
Phần 4

Hàm

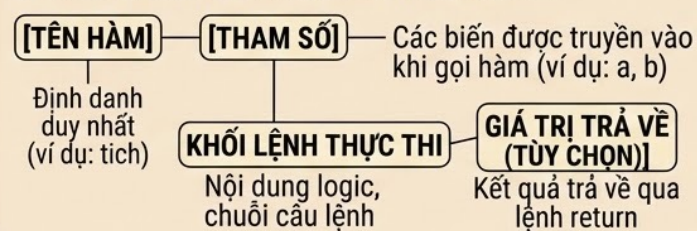
Cú pháp khai báo Hàm

SƠ ĐỒ HÀM (FUNCTION) TRONG PYTHON: CẤU TRÚC VÀ LƯỒNG DỮ LIỆU

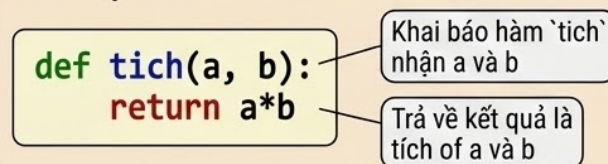
1. CÚ PHÁP KHAI BÁO HÀM



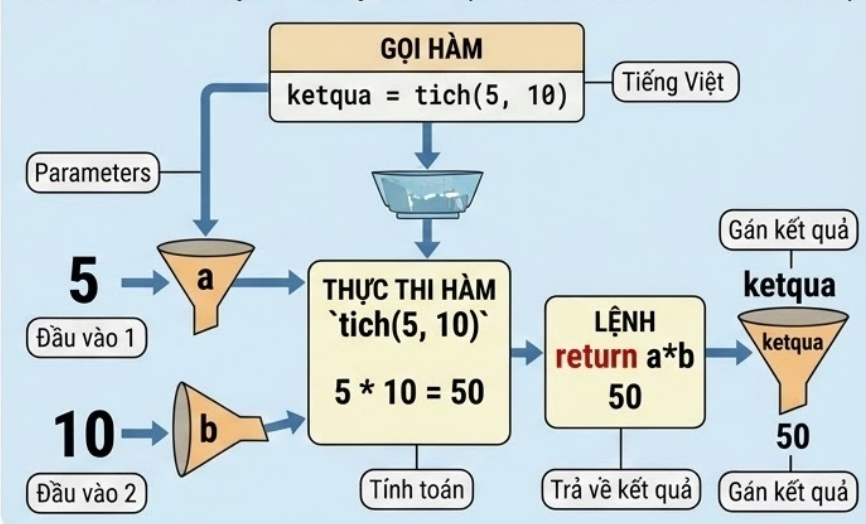
2. CÁC THÀNH PHẦN CHÍNH CỦA HÀM



3. VÍ DỤ: HÀM TÍNH TÍCH 2 SỐ



4. LƯỒNG DỮ LIỆU KHI GỌI HÀM (FUNCTION CALL DATA FLOW)



5. GIÁ TRỊ TRẢ VỀ CỦA HÀM

HÀM SỬ DỤNG 'return'	HÀM KHÔNG TRẢ VỀ
Ví dụ: <code>return a*b</code> Hàm kết thúc và trả về giá trị chỉ định. Ví dụ: trả về '50'	Ví dụ: Hàm trả về 'None' (mặc định nếu không có 'return') None Mặc định

Phần 5

Bài tập



Bài tập

1. Viết chương trình nhập số A và kiểm tra xem A có phải là số nguyên tố hay không?
2. Viết chương trình nhập hai số A và B, in ra tất cả các số nguyên tố nằm trong khoảng $[A, B]$.
3. Nhập 2 số A và B, tính và in ra màn hình ước số chung lớn nhất và bội số chung nhỏ nhất của hai số đó.
4. Nhập tọa độ 3 điểm A, B và C trên mặt phẳng 2 chiều. Hãy kiểm tra và chỉ ra hình dạng của tam giác ABC (đều, vuông, cân, vuông cân, tù, nhọn,...)